

ARQUITECTURA DE MICROSERVICIOS - PATRONES COMPOSITE Y FLYWEIGHT

Martin Cruz
Julian Gonzales
Michael Zabala

Siguiente

Inicio





DEFINICIÓN, QUÉ ES Y PARA QUÉ SIRVE

- Arquitectura de Microservicios: Estilo arquitectónico que estructura una aplicación como un conjunto de servicios independientes, débilmente acoplados y desplegables de forma autónoma, comunicándose mediante protocolos ligeros (APIs REST o gRPC). Sirve para escalar aplicaciones complejas de manera flexible y modular.

- Patrón Composite (Estructural): Permite componer objetos en estructuras de árbol para representar jerarquías parte-todo. Su propósito en microservicios es tratar a los objetos individuales y a las composiciones de servicios o componentes de manera uniforme.

- Patrón Flyweight / Peso Ligero (Estructural): Minimiza el uso de memoria compartiendo tanta data como sea posible entre objetos similares; se utiliza para soportar eficientemente grandes volúmenes de datos u objetos en sistemas distribuidos de alta concurrencia.



CARACTERÍSTICAS, CÓMO FUNCIONA Y CÓMO SE ORGANIZA

- Microservicios: Se organizan en torno a dominios de negocio (Domain-Driven Design), con bases de datos descentralizadas por servicio y despliegue automatizado mediante contenedores.
- **Composite:** Funciona mediante una interfaz común implementada tanto por objetos hojas como por contenedores. Se organiza jerárquicamente para que una petición a un servicio compuesto delegue la ejecución a sus subcomponentes de forma transparente.
- Flyweight: Separa el estado en dos partes: el estado intrínseco (compartido e independiente del contexto, almacenado en el objeto Flyweight) y el estado extrínseco (dependiente del contexto, pasado por el cliente). Se gestiona a través de una fábrica que controla el caché de instancias.



85%

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam,





OUR FACILITIES

. Representación de componentes y conexiones

Los servicios se estructuran modularmente separando las peticiones del cliente hacia la capa de enrutamiento, la cual distribuye el flujo hacia los componentes jerárquicos o hacia la factoría de instancias compartidas según corresponda en la arquitectura distribuida.

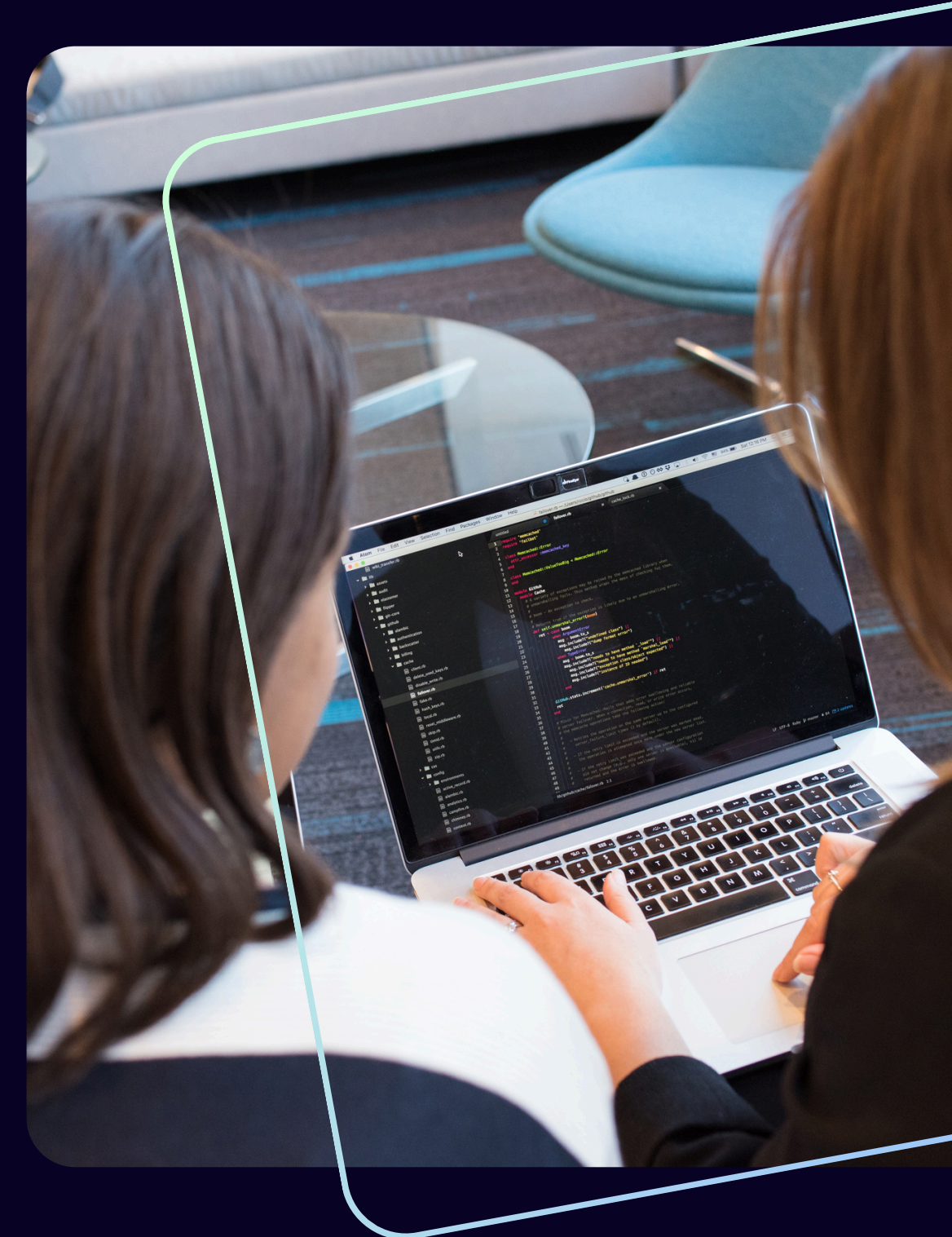
4. VENTAJAS Y DESVENTAJAS

Ventajas

- Alta modularidad, escalabilidad y mantenibilidad en sistemas distribuidos complejos.
- Optimización drástica del consumo de recursos en memoria mediante la compartición de estados (Flyweight).
- Simplificación en el manejo de estructuras anidadas o jerárquicas de servicios (Composite).

Desventajas

- Mayor complejidad inicial de diseño y arquitectura.
- Sobrecarga en la gestión y sincronización del estado extrínseco en red.
- Curva de aprendizaje elevada para mantener la cohesión de las jerarquías distribuidas.



DEMOSTRACION

En la carpeta /src de este repositorio encontrarás las implementaciones prácticas:

- `composite/`: Ejemplo en código que simula la ejecución en cascada de servicios anidados.
- `flyweight/`: Demostración de optimización de memoria compartiendo instancias comunes en alta concurrencia.

CONCLUSIÓN: ¿EN QUÉ SITUACIÓN CONVIENE UTILIZAR CADA UNO?

Conclusion

- Arquitectura de Microservicios: Conviene aplicarla en sistemas empresariales grandes, escalables y desarrollados por equipos multidisciplinarios independientes.
- Patrón Composite: Ideal cuando el sistema necesita manipular estructuras de datos jerárquicas (organigramas, menús, catálogos anidados o ensamblajes de servicios) donde el cliente interactúa de forma idéntica con elementos simples y compuestos.
- Patrón Flyweight: Se debe utilizar cuando la aplicación crea una cantidad masiva de objetos similares que amenazan con agotar la memoria RAM, permitiendo centralizar y compartir los datos comunes de solo lectura.



GRACIAS

